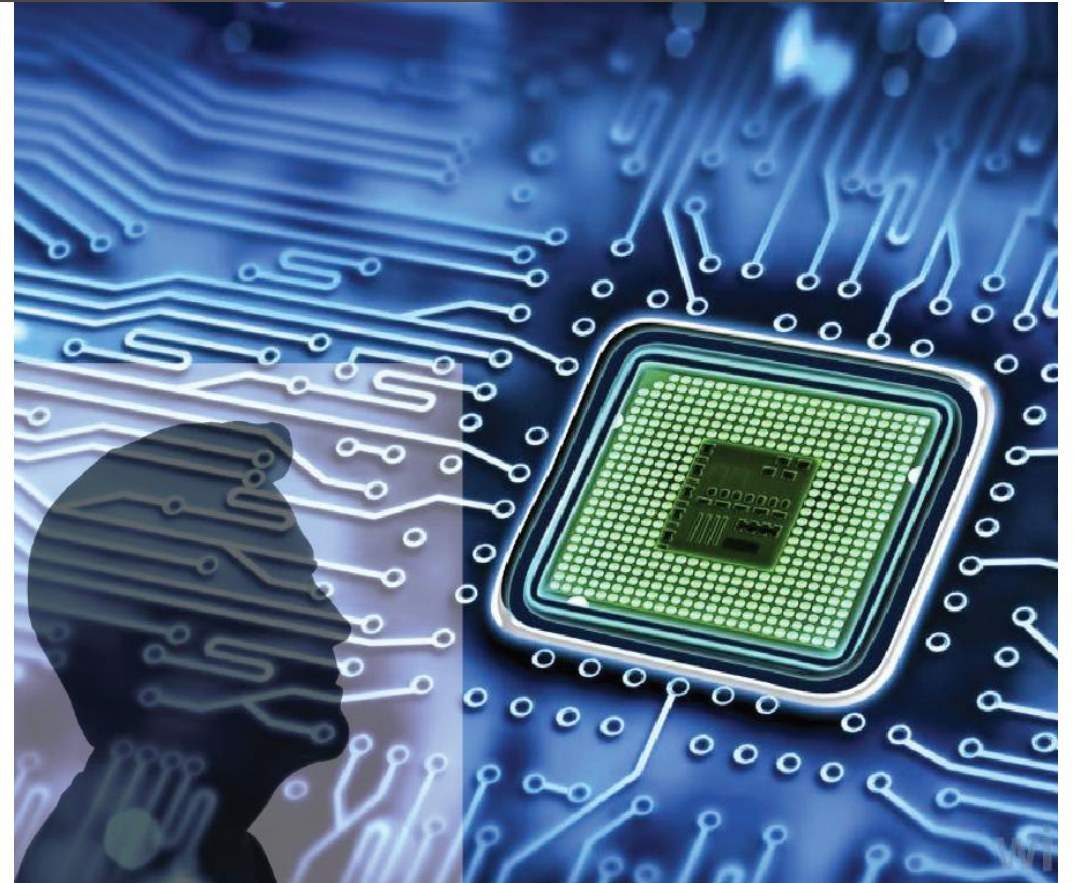


# ADVANCED DIGITAL DESIGN AND VERIFICATION

## Week-2, Lecture-2 SystemC

Sneh Saurabh  
21<sup>st</sup> August, 2025



---

# SystemC

# Modules

- **Modules** are the basic building blocks of a SystemC design hierarchy
  - Modules are classes that inherit from the `sc_module` base class.
  - As a simplification, the `SC_MODULE` macro is provided.

- Modules are organized using C++ convention
- Declaration/interface of modules is in the header (.h file)
- The implementation of the module is in the .cpp file

```
SC_MODULE(Hello_SystemC) { // declare module class
    SC_CTOR(Hello_SystemC) { // create a constructor
    }
};
```

Modules may contain:

- Constructor (mandatory)
- Ports
- Member other module instances
- Member channel instances
- Member data members
- Destructor
- Simulation process member functions
- Other member functions

# Modules: Constructor

## Role of constructor

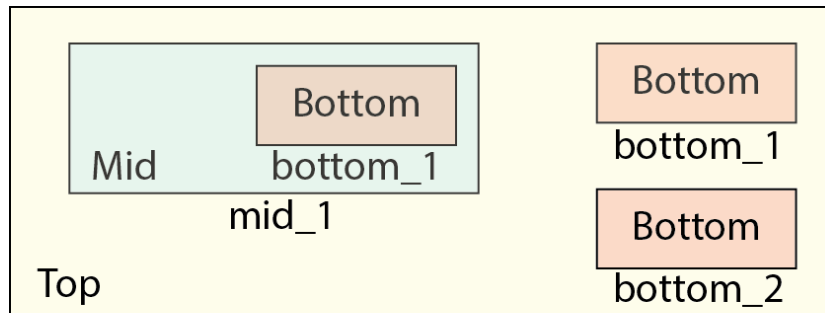
- Initializing/allocating sub-designs
- Connecting sub-designs
- Registering processes with the SystemC kernel
- Providing static sensitivity
- Miscellaneous user-defined setup

- SystemC provides the macro:  
**SC\_CTOR()**

```
SC_MODULE(module_name) {  
    SC_CTOR(module_name) :  
        Initialization // C++ initialization list  
    {  
        Subdesign_Allocation  
        Subdesign_Connectivity  
        Process_Registration  
        Miscellaneous_Setup  
    }  
};
```

# Modules and Design Hierarchy

- Instances of modules are created as member data (objects) of the parent modules



Top

```
Top.mid_1.bottom_1
Top.bottom_1
Top.bottom_2
```

- Pointers to the submodules can also be used to create instances
  - new and delete operators needed to create/destroy objects dynamically

```
#include <systemc.h>

SC_MODULE(Bottom) { // leaf level module
    SC_CTOR(Bottom) { cout << name() << endl; }
};

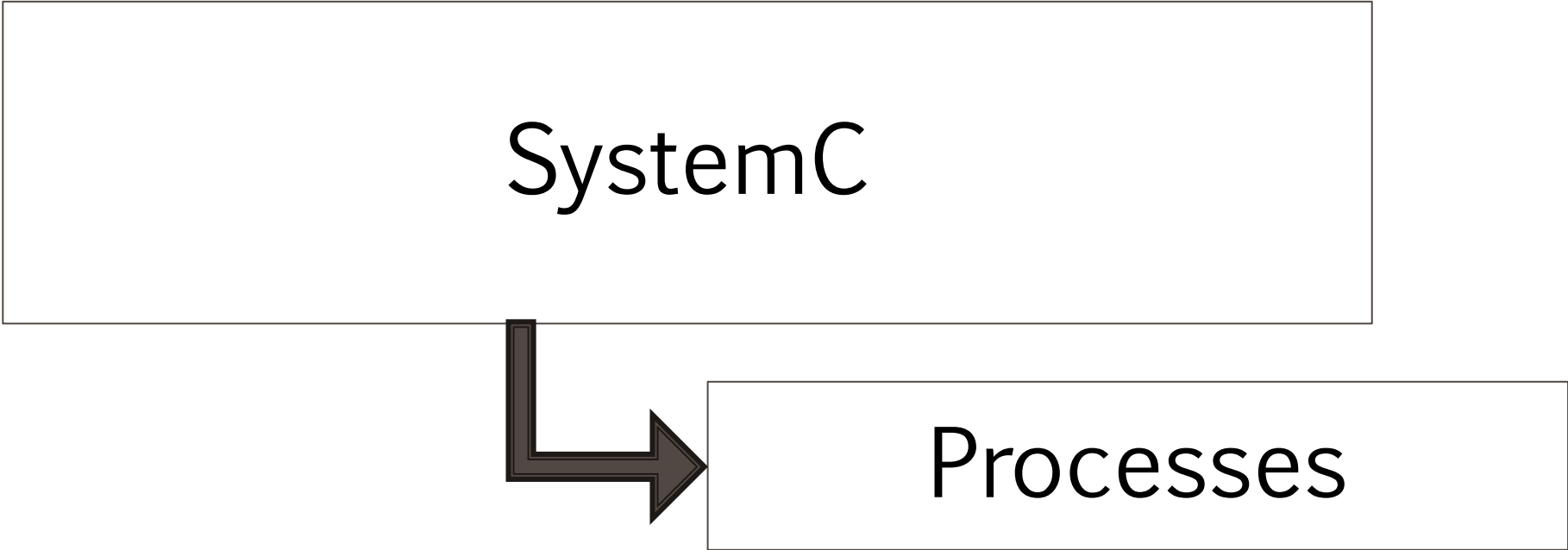
SC_MODULE(Mid) { // mid-level module
    Bottom bottom_1;
    SC_CTOR(Mid)
    : bottom_1("bottom_1") {} // create instance
};

SC_MODULE(Top) { // mid-level module
    Mid mid_1;
    Bottom bottom_1;
    Bottom bottom_2;
    SC_CTOR(Top)
    : mid_1("mid_1"),
      bottom_1("bottom_1"),
      bottom_2("bottom_2") {} // create instances
};

int sc_main(int sc_argc, char* sc_argv[]) {
    //create an instance of the top-level module
    Top Top("Top");
    sc_start(); // invoke the simulator
    return 0;
}
```

---

---



```
graph TD; A[SystemC] --> B[Processes]
```

SystemC

Processes

# Simulation Processes

- **Simulation Kernel:** it schedules the execution of the processes during simulation
  - Works as a **cooperative multitasking** (non-preemptive) kernel [cannot force a running process to return control]

- **Simulation Processes:** member functions of the **sc\_module** class that are **registered** with the simulation kernel
  - Two types: thread and method
  - Has no argument and no return value

|                         | Thread                                                                | Method                                                                                 |
|-------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Registration            | SC_THREAD                                                             | SC_METHOD                                                                              |
| Suspension              | Allowed to suspend using <b>wait</b>                                  | Allowed to continue using <b>next_trigger</b>                                          |
| Behavior                | Similar to software thread: moves from running state to waiting state | Similar to normal function: time does not change within the function from start to end |
| Variables between calls | Persistent                                                            | Lost                                                                                   |

# Simulation Processes: Syntax

```
#include <systemc.h>
SC_MODULE(two_processes) {
    void my_thread(void); // thread process
    void my_method(void); // method process
    SC_CTOR(two_processes) {
        SC_THREAD(my_thread); // register process
        SC_METHOD(my_method); // register process
    }
};

void two_processes::my_thread(void) {
    while (true) {
        cout << "In thread " << "at time: " << sc_
time_stamp() << endl;
        wait(500, SC_MS);
    }
}

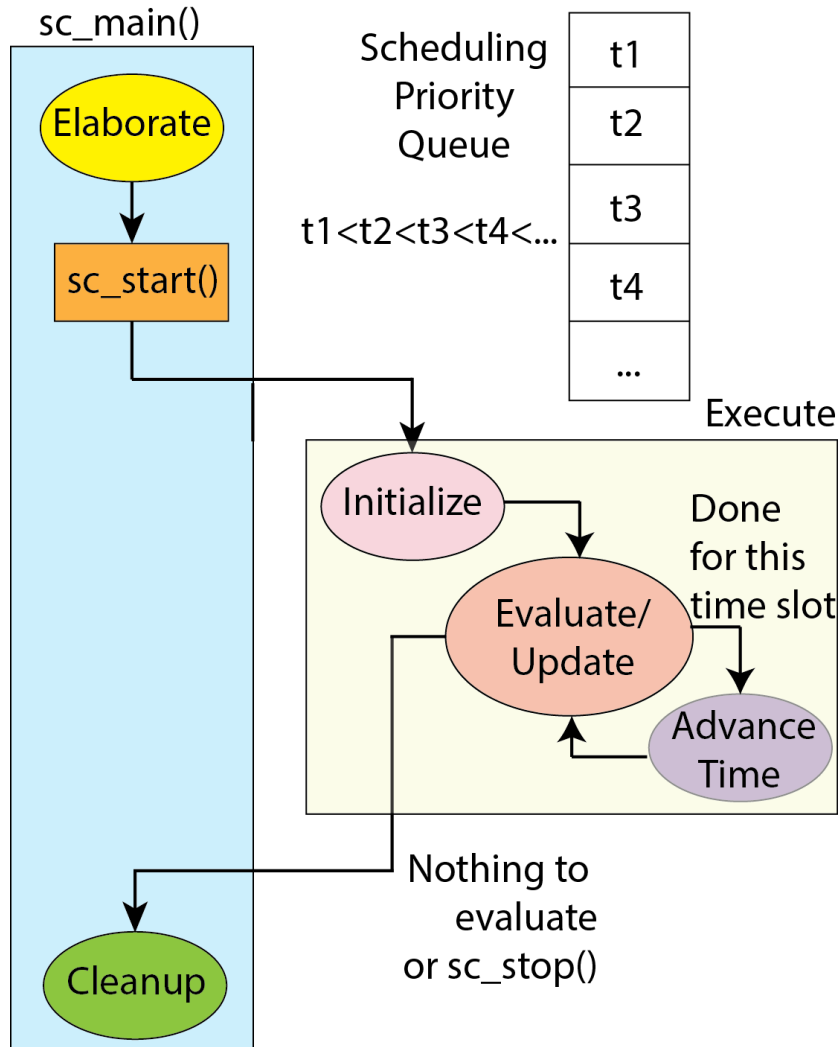
void two_processes::my_method(void) {
    cout << "In method " << "at time: " << sc_
time_stamp() << endl;
    next_trigger(200, SC_MS);
}

int sc_main(int sc_argc, char* sc_argv[]) {
    two_processes two_processes_i("two_processes_i");
    sc_start(1500, SC_MS); // invoke the simulator
    return 0;
}
```

- my\_thread and my\_method have been registered
- my\_thread has an infinite loop
  - Waits for 500ms before the next execution
- my\_method is triggered every 200 ms
- Simulation ends at 1500 ms



# Simulation in SystemC (1)



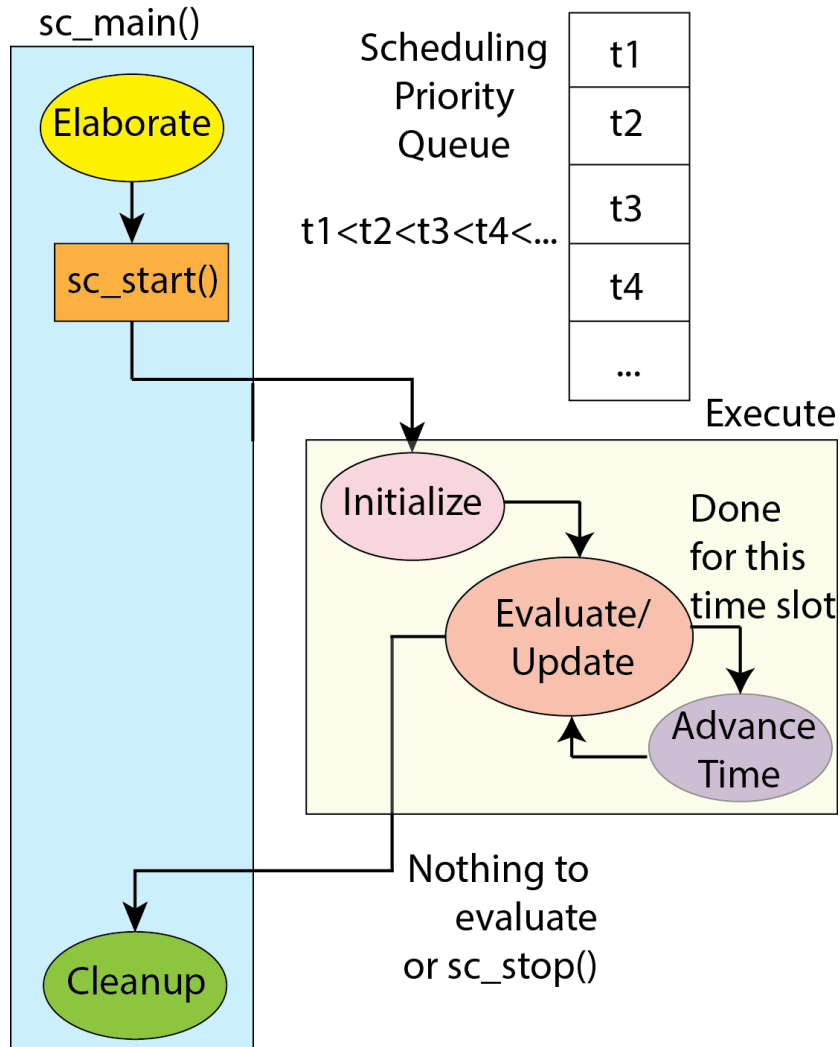
**Initialize:** identifies all simulation processes and places them in runnable/waiting sets

- Some processes may not be required to be run in the beginning: use `dont_initialize()` while registering

**Evaluate/Update:**

- All runnable processes are run **one at a time**
- There is no priority among runnable processes for a given time
  - Any process can be chosen to run first
- Each process runs until either it executes a `wait(delay)` or a return (reach the end of function).
- On execution of `wait(delay)`, that process is appropriately stored in the scheduling priority queue

# Simulation in SystemC (2)



## Evaluate/Update:

- After all runnable processes are executed:
  - Simulated time is advanced to the closest time with a scheduled event
  - Move processes waiting for that particular time into the runnable set
- This process continues until:
  - Runnable set becomes empty (nothing to evaluate)
  - `sc_stop()` is encountered
  - Maximum simulation time is reached
- Simulation stops and the cleanup phase starts

# Simulation Processes: Example

```
#include <systemc.h>
SC_MODULE(two_processes) {
    void my_thread(void); // thread process
    void my_method(void); // method process
    SC_CTOR(two_processes) {
        SC_THREAD(my_thread); // register process
        SC_METHOD(my_method); // register process
    }
};

void two_processes::my_thread(void) {
    while (true) {
        cout << "In thread " << "at time: " << sc_
time_stamp() << endl;
        wait(500, SC_MS);
    }
}

void two_processes::my_method(void) {
    cout << "In method " << "at time: " << sc_
time_stamp() << endl;
    next_trigger(200, SC_MS);
}

int sc_main(int sc_argc, char* sc_argv[]) {
    two_processes two_processes_i("two_processes_i");
    sc_start(1500, SC_MS); // invoke the simulator
    return 0;
}
```

- my\_thread and my\_method have been registered
- my\_thread has an infinite loop
  - Waits for 500ms before the next execution
- my\_method is triggered every 200 ms
- Simulation ends at 1500 ms

```
In method at time: 0 s
In thread at time: 0 s
In method at time: 200 ms
In method at time: 400 ms
In thread at time: 500 ms
In method at time: 600 ms
In method at time: 800 ms
In method at time: 1 s
In thread at time: 1 s
In method at time: 1200 ms
In method at time: 1400 ms
```

- This example shows the dynamic triggering of processes

# Events

---

- Events allow interactions between processes and channels
- An event is something that happens at a specific instant in time.
- SystemC uses the `sc_event` class to model events
- An event has no value and no duration

Two operations on an event:

- Cause it to happen: member function `notify()`
  - Notification can occur within a simulation process or as a result of activity in a channel
- Catch event: using `wait(event)`

# Events: Example

```
SC_MODULE(two_processes) {  
    void give(void); // thread process  
    void take(void); // thread process  
    sc_event given; // event of giving completion  
    sc_event taken; // event of taking completion  
    SC_CTOR(two_processes) {  
        SC_THREAD(give); // register process  
        SC_THREAD(take); // register process  
    }  
};
```

- Two threads **give** and **take** registered
- Two events **given** and **taken** declared
- Thread **take** waits for the event **given**
- Thread **give** waits for the event **taken**

```
void two_processes::give(void) {  
    while (true) {  
        given.notify(500, SC_MS);  
        wait(taken);  
        cout << "Taken " << "at time: " << sc_time_stamp() << endl;  
    }  
}  
  
void two_processes::take(void) {  
    while (true) {  
        wait(given);  
        cout << "Given " << "at time: " << sc_time_stamp() << endl;  
        taken.notify(100, SC_MS);  
    }  
}
```

```
int sc_main(int sc_argc, char* sc_argv[]) {  
    two_processes two_processes_i("two_processes_i");  
    sc_start(1500, SC_MS); // invoke the simulator  
    return 0;  
}
```

```
Given at time: 500 ms  
Taken at time: 600 ms  
Given at time: 1100 ms  
Taken at time: 1200 ms
```

# Dynamic and Static Sensitivity

| Dynamic Sensitivity                                                                                         | Static Sensitivity                                                                                          |
|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Allows a simulation process to change its sensitivity on the fly                                            | Sensitivity defined during process registration (within constructor) and handled during elaboration         |
| Invoked when <code>wait(arg)</code> defined for thread or <code>next_trigger(arg)</code> defined for method | Invoked when <code>wait(void)</code> defined for thread and no <code>next_trigger</code> defined for method |

- We have already seen examples of dynamic sensitivity
- Static sensitivity is established for each process immediately after its process registration.
- **Events** may be added to a process's static sensitivity list, which is initially empty
  - Using the insertion operator (<<) on a special `sc_module` data member called `sensitive`

# Static Sensitivity: Example

```
#include <systemc.h>

SC_MODULE(ClkGen) {
    void toggle(void); // method process
    sc_event edge; // event of edge
    bool clk; // a data member
    SC_CTOR(ClkGen) : clk(false) {
        SC_METHOD(toggle); // register process
        sensitive << edge; // define static sensitivity
    }
};

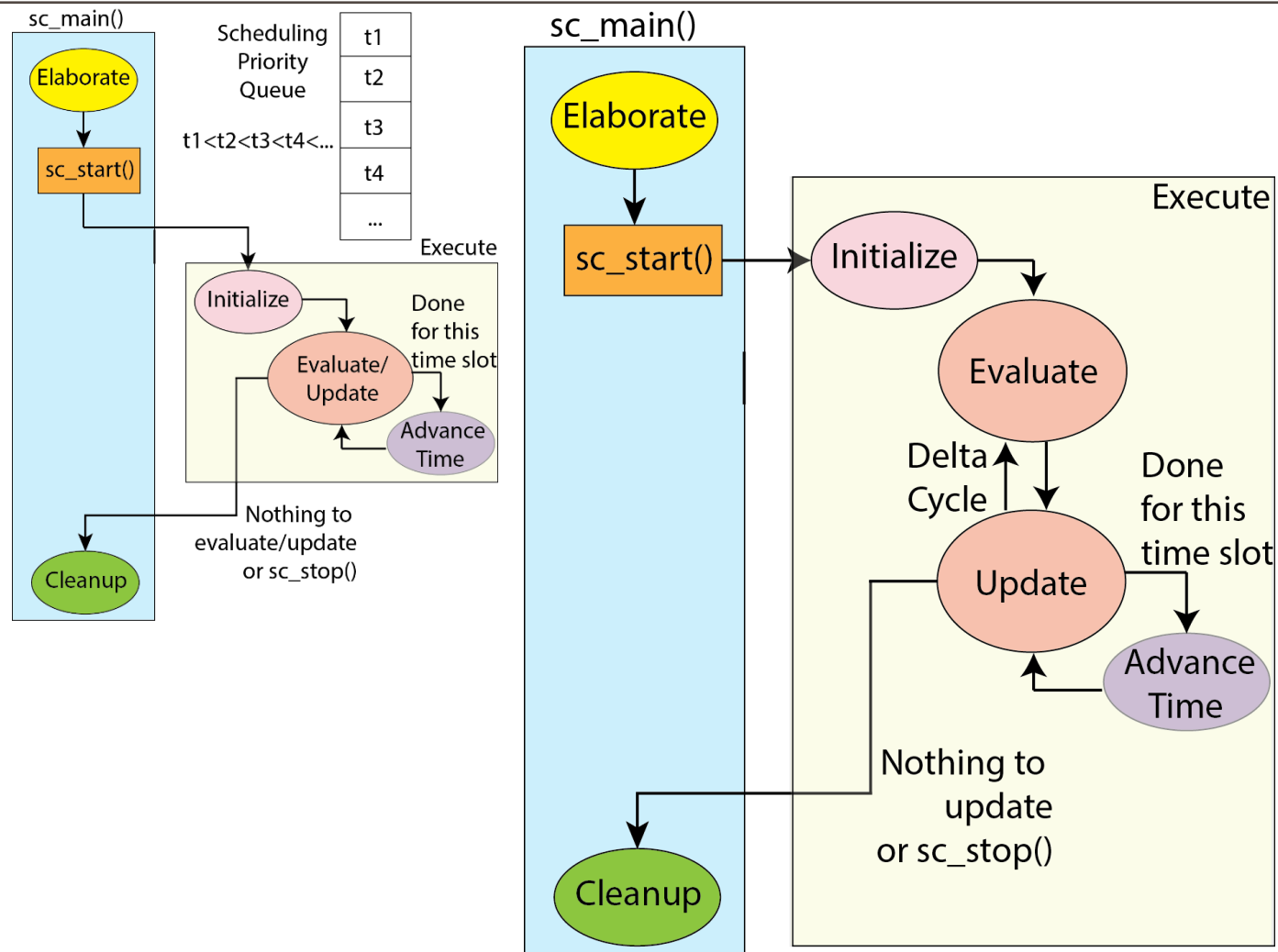
void ClkGen::toggle(void) {
    cout << "clk " << clk << " time: " << sc_time_stamp() << endl;
    edge.notify(1, SC_NS);
    clk = !clk;
}

int sc_main(int sc_argc, char* sc_argv[]) {
    ClkGen ClkGen_i("ClkGen_i");
    sc_start(6, SC_NS); // invoke the simulator
    return 0;
}
```

- Defined static sensitivity for the method
- No next trigger
- Hence static sensitivity used when the event is notified

```
Clk 0 time: 0 s
Clk 1 time: 1 ns
Clk 0 time: 2 ns
Clk 1 time: 3 ns
Clk 0 time: 4 ns
Clk 1 time: 5 ns
```

# SystemC Signal (1)



- To mimic the operation of hardware, the simulation engine has two execution steps in the same time slot: **Evaluate and Update**

- Simulator can go from Evaluate to Update, (and back): This is called the **delta cycle**
- Time does not advance in the delta cycle

- Delta cycles are used in SystemC `sc_signal`



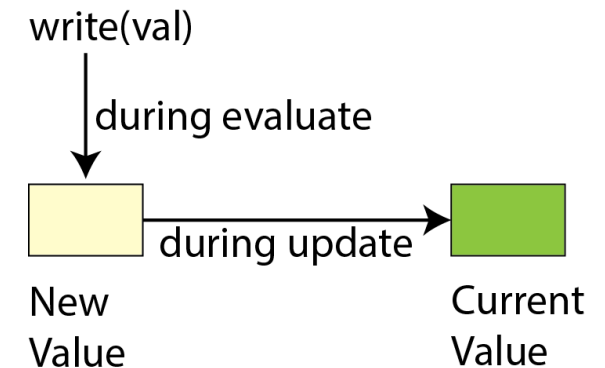
# SystemC Signal (2)

- SystemC provides a primitive channel `sc_signal<T>` to model evaluate-update paradigm
  - It has member function `write()` which is the evaluate phase of the delta cycle
  - `write(T)` is identical to Verilog's `reg` with a non-blocking assignment
  - `read()` fetches the current value of the channel

Example:

```
sc_signal<int> val;  
val.write(5);  
int current_val = val.read();
```

- Every channel can be considered to have two storage locations: the current value and the new value



- The current value remains unchanged until the end of the update phase.
- If a process writes to an evaluate-update channel and then immediately (i.e., without suspending) accesses the channel, it will find the current value unchanged

# Signal: Example

```
SC_MODULE(signal_processes) {
    void my_thread(void); // process
    bool s;
    sc_signal<bool> sig;
    SC_CTOR(signal_processes) {
        SC_THREAD(my_thread); // register process
        s = true;
        sig.write(true);
    }
};

//FILE: signal_processes.cpp
void signal_processes::my_thread(void) {
    cout << "Initial: " << "s=" << s << " sig=" <<
        sig.read() << " T=" << sc_time_stamp() << endl;
    s = false;
    sig.write(false);
    cout << "Before delta: " << "s=" << s << " sig=" <<
        sig.read() << " T=" << sc_time_stamp() << endl;
    wait(SC_ZERO_TIME);
    cout << "After delta: " << "s=" << s << " sig=" <<
        sig.read() << " T=" << sc_time_stamp() << endl;
}
```

- Normal variable updated immediately
- sc\_signal data updated only after the end of the delta cycle

```
Initial: s=1 sig=1 T=0 s
Before delta: s=0 sig=1 T=0 s
After delta: s=0 sig=0 T=0 s
```

# Signal: Other member functions

---

## Other member functions of `sc_signal<T>`

- `value_changed_event()`: returns reference to an event [this event gets notified when an update causes a change in value]
- `bool event(void)`: returns true/false to see if the channel issued an event in the immediately previous delta-cycle

## Special member functions for `sc_signal<bool>` and `sc_signal<sc_logic>`

- `posedge_event()`: returns reference to an event [this event occurs on any transition to SC\_LOGIC\_1 in the immediately previous delta-cycle]
- `negedge_event()`: returns reference to an event [this event occurs on any transition to SC\_LOGIC\_0 in the immediately previous delta-cycle]

`sc_buffer<T>` is a predefined channel derived from `sc_signal<T>`

- `value_changed_event()`: returns reference to an event whenever buffer is written, rather than when value is changed

# Events: Immediate and Delayed Notification

---

## Immediate Notification

- `notify(void)`
- Processes waiting for the event are immediately moved from the wait pool into the ready pool for execution (**before the update**)

## Delayed Notification

- `notify(SC_ZERO_TIME)`
- Processes waiting for a delayed notification will be executed on the next delta cycle (**after the update**)

# Immediate and Delayed Notification: Example

```
SC_MODULE(signal_processes) {
    void sender(void); // process
    void receiver_immediate(void); // process
    void receiver_delayed(void); // process
    sc_signal<bool> sig;
    sc_event immediate;
    sc_event delayed;
    SC_CTOR(signal_processes) {
        SC_THREAD(sender); // register process
        SC_THREAD(receiver_immediate); // register process
        SC_THREAD(receiver_delayed); // register process
        sig.write(false);
    }
};
```

- Two threads wait on events
- `receiver_immediate()` gets triggered by immediate notification
- `receiver_delayed()` gets triggered by delayed notification

```
Initial:  sig=0 T=0 s
Immediate: sig=0 T=0 s
Delayed:  sig=1 T=0 s
```

```
void signal_processes::sender(void) {
    wait(SC_ZERO_TIME);
    cout << "Initial: " << " sig=" <<
        sig.read() << " T=" << sc_time_stamp() << endl;
    sig.write(true);
    delayed.notify(SC_ZERO_TIME);
    immediate.notify();
}

void signal_processes::receiver_immediate(void) {
    wait(immediate);
    cout << "Immediate: " << " sig=" <<
        sig.read() << " T=" << sc_time_stamp() << endl;
}

void signal_processes::receiver_delayed(void) {
    wait(delayed);
    cout << "Delayed: " << " sig=" <<
        sig.read() << " T=" << sc_time_stamp() << endl;
}
```

- A process gets triggered by an event only when it is waiting for that event
  - To allow both receiver processes to be waiting before the event `wait(SC_ZERO_TIME)` is added in `sender()`

# References

---

- Black, David C., and Jack Donovan, eds. SystemC: From the ground up. Boston, MA: Springer US, 2004.